

this chapter for more details.) Behind the scenes, Python calls the `__add__()` method with `42` for the `other` parameter, which also returns `NotImplemented`, causing Python to raise a `TypeError`.

Although we shouldn't be able to add integers to or subtract them from `WizCoin` objects, it would make sense to allow code to multiply `WizCoin` objects by positive integer amounts by defining a `__mul__()` dunder method. Add the following to the end of `wizcoin.py`:

```
--snip--
def __mul__(self, other):
    """Multiplies the coin amounts by a non-negative integer."""
    if not isinstance(other, int):
        return NotImplemented
    if other < 0:
        # Multiplying by a negative int results in negative
        # amounts of coins, which is invalid.
        raise WizCoinException('cannot multiply with negative integers')

    return WizCoin(self.galleons * other, self.sickles * other, self.knuts *
other)
```

This `__mul__()` method lets you multiply `WizCoin` objects by positive integers. If `other` is an integer, it's the data type the `__mul__()` method is expecting and we shouldn't return `NotImplemented`. But if this integer is negative, multiplying the `WizCoin` object by it would result in negative amounts of coins in our `WizCoin` object. Because this goes against our design for this class, we raise a `WizCoinException` with a descriptive error message.

NOTE

You shouldn't change the `self` object in a numeric dunder method. Rather, the method should always create and return a new object. The `+` and other numeric operators are always expected to evaluate to a new object rather than modifying an object's value in-place.

Enter the following into the interactive shell to see the `__mul__()` dunder method in action:

```
>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10) # Create a WizCoin object.
>>> purse * 10 # Multiply the WizCoin object by an integer.
```